Agent Evals

A Track → Measure → Improve loop for AI agents · Project report · Dev Patel · October 2026

1. Summary

This project is a small, working framework that tests an AI agent, scores its answers, and uses the scores to improve the agent automatically. It follows the three-step idea **track, measure, improve**: every answer is recorded, three metrics are calculated (**accuracy, hallucination, quality**), and the weakest metric decides which fix is applied to the agent's system prompt before the next round. The loop repeats until targets are met. The code is a single Python file with unit tests, packaged as a GitHub-ready repository.

On the included demo suite the loop took the agent from **83% accuracy, 33% hallucination and 70% quality** to **100%, 0% and 100%** in three improvement rounds.

2. Why we built it

Most agent demos are judged by eye: someone tries a few questions and decides it “looks fine”. That fails in practice because agents can be confidently wrong, invent facts that are not in their source material, or ramble. Changing a prompt to fix one problem can quietly break another. Evals solve this by turning behaviour into numbers that can be compared before and after every change. We chose this project for three reasons:

- **It is a real engineering need.** Anyone shipping an agent has to know whether version two is better than version one.
- **It is small enough to finish and explain.** A complete loop in one file is easier to read, test and demonstrate than a large platform.
- **It shows the full cycle.** Many portfolios show an agent; few show the measurement and feedback around it.

3. What we built

Component	What it does	Why it is built this way
Test suite	Six questions about a fixed refund-policy text, each with the facts a correct answer must contain.	A fixed source text makes “correct” and “invented” objectively checkable.
Agent	A function *agent(prompt, question, context)*. A mock agent runs offline; a live agent calls Claude.	One interface means any agent can be dropped in without touching the evaluators.
Evaluators	Three functions score each answer: accuracy, hallucination, quality.	Separate metrics show *what kind* of failure occurred, not just that one did.
Improver	Finds the worst metric and adds a matching rule to the system prompt.	A deterministic rule per metric keeps each change explainable.
History log	Saves prompt, scores and every answer for each round to JSON.	The “track” step: results can be audited and charted later.

4. System design

The core is a closed loop. The agent answers the test questions, the evaluator scores the answers on three axes, and the improver rewrites the system prompt based on the weakest score. The new prompt goes back to the agent for the next round. Figure 1 shows the loop; it mirrors the idea that evaluation is not a one-off test but a continuous cycle.

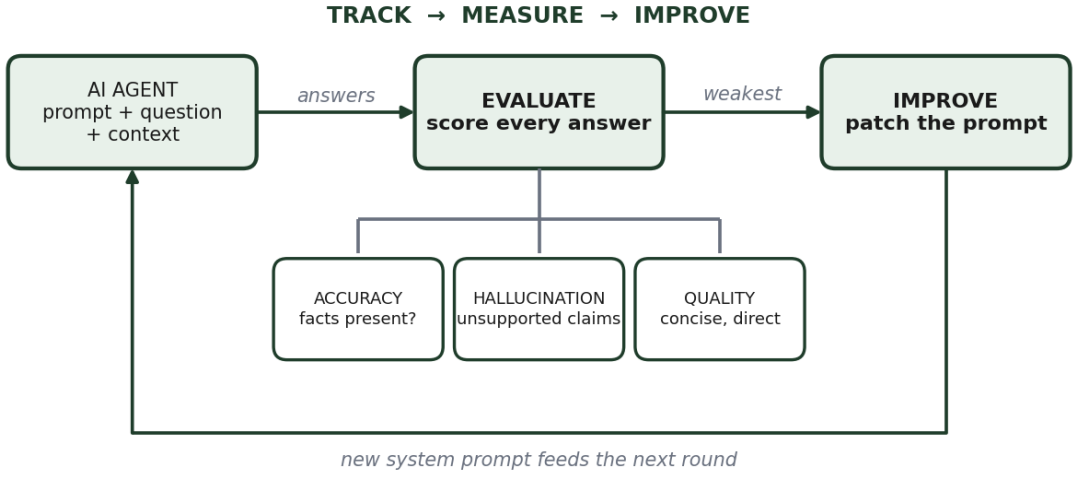


Figure 1. The core loop: the agent is evaluated on three metrics and the weakest one drives the next prompt change.

Figure 2 shows what happens inside one round. The test suite and the current system prompt feed the agent. Each answer is passed through all three evaluators independently, and the per-metric averages are aggregated and written to the history file. **Why this design:** evaluators only see the answer, the question and the context, so they never depend on how the agent works internally. That is what lets the same harness test a mock agent today and a real model tomorrow.

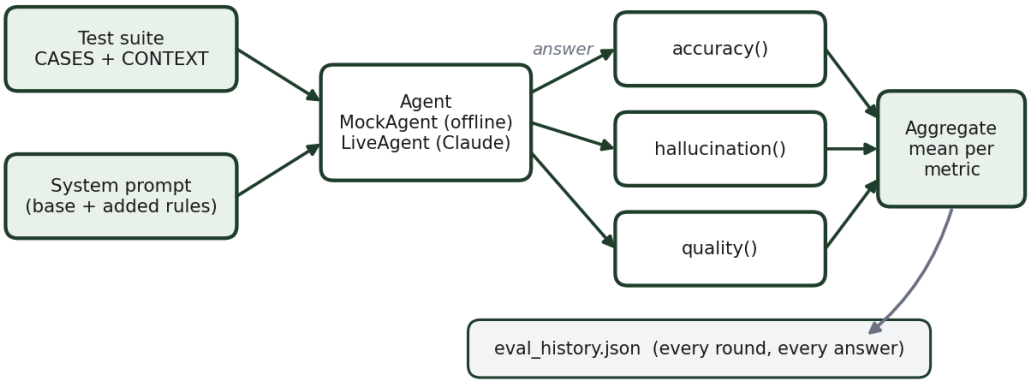


Figure 2. Data flow inside one round, from inputs to the saved history.

5. Why an offline mock agent

The mock agent is deterministic and gets better as rules are added to its prompt: without the grounding rule it appends an invented sentence on some answers, without the concise rule it adds filler, and without the double-check rule it gets one fact wrong. This is deliberate. It lets anyone run the project with no API key or cost, makes results identical on every run (which is what unit tests need), and proves the *harness* works, since the harness must detect a problem before it can fix one. Running with `--live` swaps in a real model for the real test.

6. How each metric works

Figure 3 breaks down the three evaluators. We chose simple, transparent rules over a model-based judge for the first version because they are free, instant, repeatable and easy to explain line by line.

- **Accuracy** checks that every expected fact appears in the answer (case-insensitive). It is strict on purpose: an answer with the right number but a missing day still scores zero, which forces the agent to be complete.
- **Hallucination** measures the share of sentences that are not supported by the source text. A sentence is unsupported when fewer than 60% of its content words appear in the context or the question. Filler and hedging sentences are removed first, because they are not factual claims; the quality metric penalises them instead. This avoids punishing the same flaw twice.
- **Quality** starts at 1.0 and subtracts for answers over 40 words (–0.4), hedging such as “I think” or “maybe” (–0.3) and filler closers (–0.2). It rewards what a customer wants: a short, direct answer.

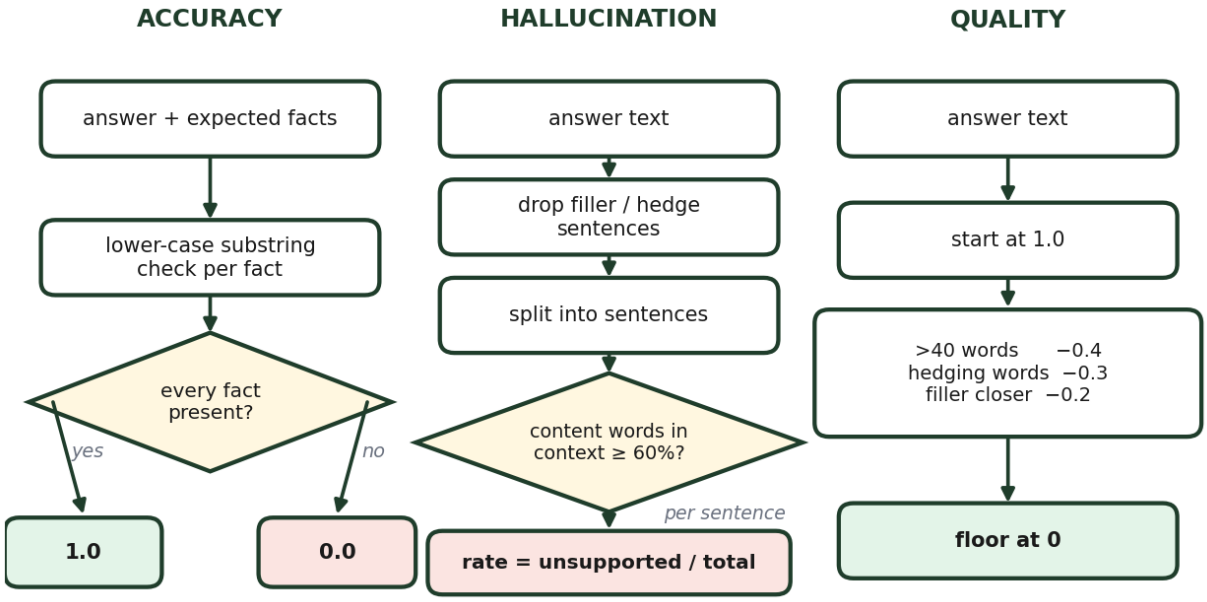


Figure 3. Flowcharts of the three evaluators.

7. The improvement algorithm

Figure 4 is the decision logic. After each round the harness checks the targets (accuracy ≥ 95%, hallucination ≤ 5%, quality ≥ 90%). If they are met it stops. Otherwise it converts the three scores to a common “badness” scale, ranks them, and applies the rule for the worst metric that has not been used yet. If every useful rule is already in the prompt it stops rather than looping forever; a hard cap of five rounds is a second safety net.

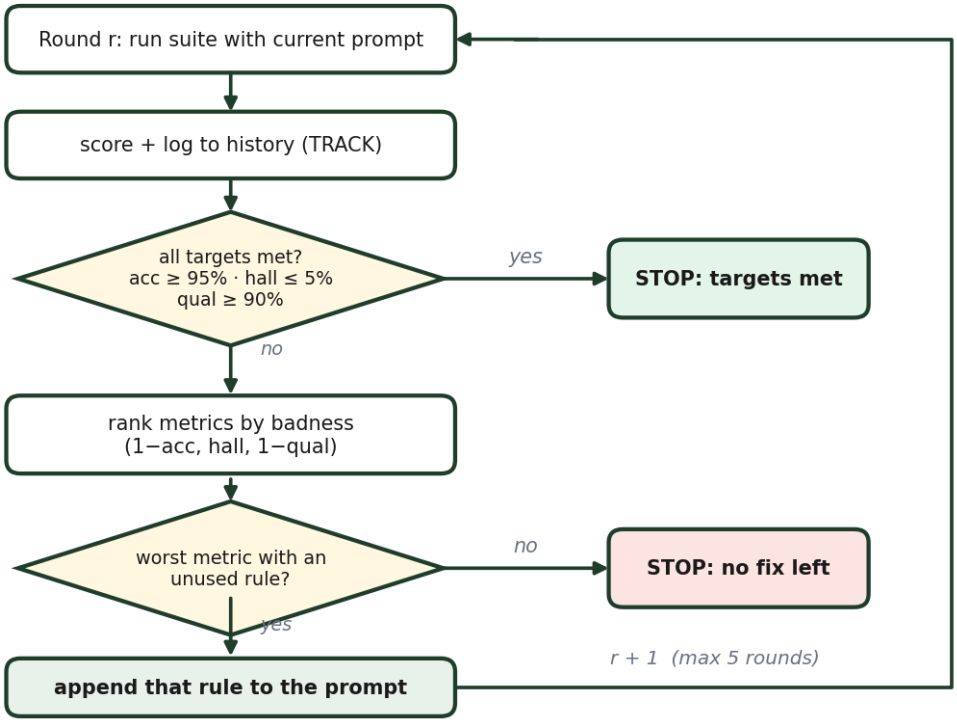


Figure 4. Decision flow of the improvement loop, including both stop conditions.

Why one change per round: if several edits are made at once, a score change cannot be attributed to any of them. Changing one thing and re-measuring shows exactly which rule helped. **Why worst-first:** it targets the biggest gap and avoids spending effort polishing a metric that is already good.

8. Results

Figure 5 and the table below show the saved demo run. Each rule fixed the metric it was written for, and no earlier gain was lost. These numbers come from the offline mock agent, so they demonstrate that the harness detects problems and drives improvement; they are not a claim about any real model's performance.

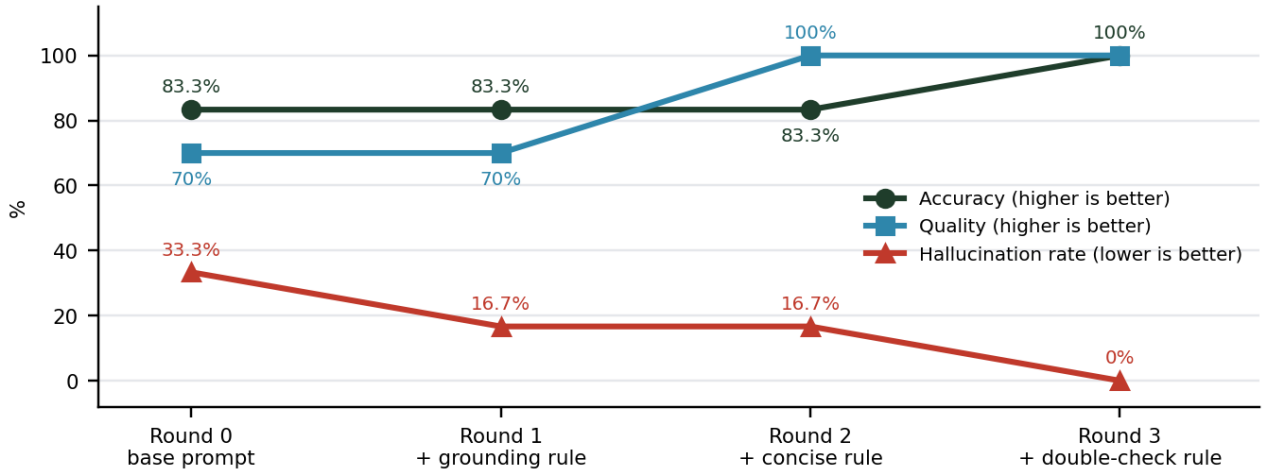


Figure 5. Metrics across rounds; each label under the axis shows the rule added before that round.

Round	Rule added before the round	Accuracy	Hallucination	Quality
0	None (base prompt)	83.3%	33.3%	70%
1	Answer only from the provided context	83.3%	16.7%	70%
2	Be concise: one or two direct sentences	83.3%	16.7%	100%
3	Double-check numbers and days against the context	100%	0%	100%

9. A bug the evals caught in our own code

The first run reported 58% hallucination and then stopped early with “no new rule to add”. Two real problems were behind it. First, the hallucination check counted filler sentences such as “please let me know if anything else is unclear” as invented facts. We fixed it by removing filler before checking. Second, the improver kept choosing the same worst metric even after its rule was already applied, so it never moved on. We fixed it by choosing the worst metric *that still has an unused rule*. Measuring the evaluator itself, not only the agent, is why the final numbers can be trusted. Four unit tests now guard this behaviour.

10. Limitations and next steps

- **Heuristic scoring.** Word overlap cannot catch a fluent but wrong paraphrase. The next version should add an LLM-as-judge for hallucination and quality.
- **Small test set.** Six cases demonstrate the loop but are not statistically strong. The plan is to load larger JSONL datasets.
- **Fixed rules.** Improvements come from a lookup table. A stronger version would have a model propose and test new prompt edits.
- **No regression history across code changes.** Tracking scores per commit would turn this into a continuous-integration check for agents.

11. Conclusion

Agent Evals shows that improving an agent can be an engineering process rather than guesswork: define what good means, measure it automatically, change one thing, and measure again. The design keeps the agent, the metrics and the improver separate, so each can be upgraded independently, and the repository includes tests, sample output and documentation so others can reproduce every result.